

우주 침략자 게임 개선 보고서

2021-17385

산업공학과 김예원

1. Q.1과 Q.4의 그래픽 장치 처리 방식

Q.04 폴더에는 RTL이 없다. Q.4 답변의 예제 코드에서 처음으로 그래픽 LCD 버스 신호(lcd_write_en, lcd_x_addr)가 등장하고, 이 방식으로 작성된 RTL이 Q.5의 space_invaders_engine_glcd.v이다. 따라서 Q.01과 Q.05의 RTL을 비교하였다.

```
// Q.01 space_invaders_engine.v          // Q.05 space_invaders_engine_glcd.v
input wire [9:0] vga_x,                 output reg      lcd_write_en,
input wire [9:0] vga_y,                 output reg [6:0] lcd_x,
input wire      video_on,               output reg [5:0] lcd_y,
output reg [2:0] rgb                    output reg      lcd_data
```

[Q.01] VGA 제어기 —(vga_x, vga_y, video_on)—▶ 게임 엔진 —(rgb)—▶ 모니터
(주소 카운터: 외부) (조합회로: 좌표 → 색)

[Q.05] 게임 엔진 —(lcd_write_en, lcd_x, lcd_y, lcd_data)—▶ LCD(GRAM) —▶ 화면
(주소 카운터 scan_x, scan_y: 엔진 내부)

표 1. Q.01과 Q.05(Q.4 방식)의 그래픽 처리 비교

항목	Q.01: VGA 방식	Q.05: 그래픽 LCD 방식
좌표 신호 방향	vga_x, vga_y 입력	lcd_x, lcd_y 출력
화면 주소 카운터	없음 (외부 VGA 제어기가 생성)	있음 (scan_x, scan_y, 148~165줄)
엔진의 역할	주어진 좌표의 색을 응답 (슬레이브)	주소와 화소 값을 LCD에 기록 (버스 마스터)
출력 회로	always @(*) 조합회로	always @(posedge clk) 순차회로 (주소-데이터 동시 갱신)
프레임 버퍼	없음. 주사 시점에 색을 출력해야 함	LCD 내부 GRAM에 저장
시간 제약	화소 클럭(640×480, 약 25MHz)에 동기	LCD가 값을 보존하므로 변경된 화소만 기록 가능
해상도 / 색	640×480 / 3비트	128×64 / 1비트
좌표 비트 폭	10비트 ×2	7비트 + 6비트
테스트벤치 역할	좌표 생성 (Q.02: 800×525 이중 for)	엔진 출력을 받아 화면에 표시 (Q.05: DPI-C + SDL2)

Q.01의 게임 엔진은 화면 좌표(vga_x, vga_y)를 밖에서 받아서, 그 위치의 색만 알려 주는 구조이다. 화면을 왼쪽 위부터 한 칸씩 훑는 일은 엔진 밖의 VGA 제어기가 한다. 반면 Q.05의 엔진은 안에 있는 카운터(scan_x, scan_y)로 화면을 직접 훑으면서, "이 위치에 이 값을 써라"라고 LCD에 좌표와 화소 값을 내보낸다. 즉 좌표가 입력에서 출력으로 바뀌었다는 것은, 화면을 그리는 일

을 누가 이끄는지가 바뀌었다는 뜻이다. 또 Q.01은 화면 내용을 따로 저장하지 않지만, Q.05는 LCD가 받은 값을 메모리에 저장해 둔다. 이 차이 때문에 칩에서 필요한 입출력 핀의 구성도 달라진다.

2. Q.10 실습과 스프라이트 시현 방식 비교

2.1 실습

```
$ cd Q.10_SC_Font_Score_Life
$ make # Gemini 답변 → 새 RTL → Q.09b와 diff → Build & Run
```

- 빌드 성공. Verilator 경고 38건: WIDTHTRUNC 16, WIDTHEXPAND 13, BLKSEQ 8, UNSIGNED 1
- Q.10은 테스트벤치를 Q.09b, 화면 모델을 Q.09d에서 가져와 사용함 (창 크기는 Q.09d 파일에서 수정)

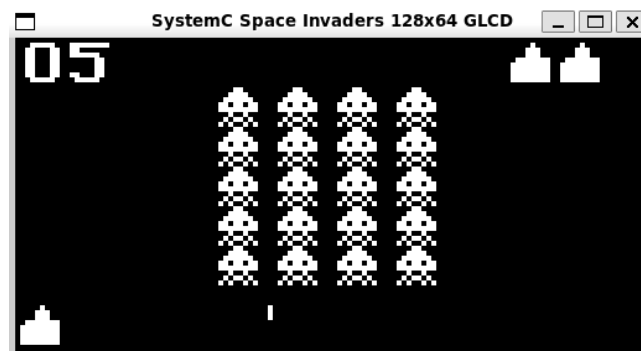


그림 1. Q.10 실행 화면. 외계인-대포가 8x8 스프라이트로 표시되고, 왼쪽 위에 점수(05), 오른쪽 위에 목숨 (대포 스프라이트 재사용)이 표시됨

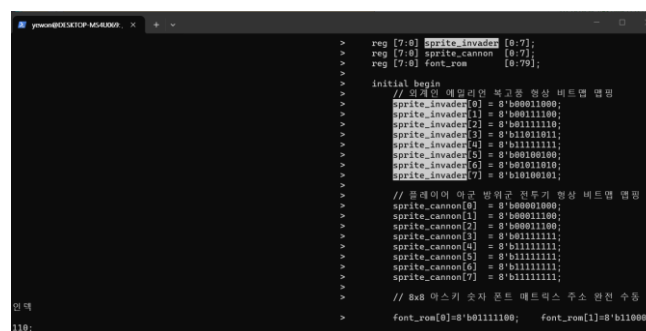


그림 2. Q.09b(왼쪽)와 Q.10(오른쪽) diff. >는 추가된 줄. initial 블록 안에서 sprite_invader, sprite_cannon, font_rom 초기화

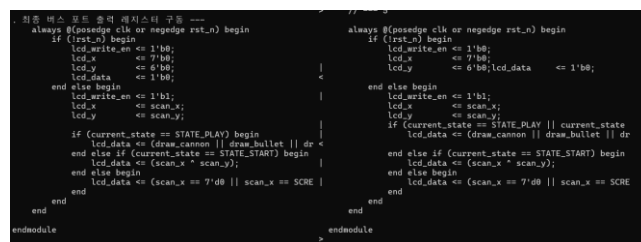


그림 3. 화면 출력부 diff. 출력 조건이 STATE_PLAY → STATE_PLAY || STATE_DIE로 변경. 오른쪽 위 lcd_y와 lcd_data 대입문이 한 줄에 연결됨

2.2 스프라이트 구현 코드

```
// 베릴로그 RTL (Q.10, 372~389줄): 주사 좌표가 외계인 영역 안이면 ROM의 비트 하나를 참조
if ((scan_x >= curr_inv_x) && (scan_x < curr_inv_x + OBJ_WIDTH) &&
    (scan_y >= curr_inv_y) && (scan_y < curr_inv_y + OBJ_HEIGHT))
    if (sprite_invader[scan_y - curr_inv_y][3'd7 - (scan_x - curr_inv_x)])
        draw_invader = 1'b1;
...
lcd_data <= (draw_cannon || draw_bullet || draw_invader || draw_inv_bullet || draw_hud_score ||
draw_hud_life);
```

```
// 아두이노 C++ (1주차 pong_Uno/04_Table_Ball_Paddle.ino)
const unsigned char ball_bitmap[] PROGMEM = { 0x18, 0x7E, 0x7E, 0xFF, 0xFF, 0x7E, 0x7E, 0x18 };
void loop() {
    drawBall(xPos, yPos, false); // 이전 위치를 검은색으로 지움
    ...                          // 위치 계산
    drawBall(xPos, yPos, true);  // 새 위치에 그림: oled.drawBitmap(x, y, ball_bitmap, 8, 8, WHITE)
    oled.display();              // 1KB 화면 버퍼를 I2C로 OLED에 전송
}
```

표 2. 스프라이트 시현 방식 비교

항목	아두이노 C++ (drawBitmap)	베릴로그 RTL (sprite_invader)
처리 기준	물체 기준: 물체 위치에 비트맵 복사 (BitBlt)	화소 기준: 현재 주사 화소에 물체가 있는지 판정
실행 방식	CPU가 명령을 순차 실행 (지우기 → 계산 → 그리기 → 전송)	모든 물체의 판정 회로가 매 클럭 병렬 동작
화면 저장	MCU SRAM의 프레임 버퍼 (128×64 = 1KB)	엔진 내부 프레임 버퍼 없음
이전 그림 처리	검은색으로 다시 그려서 지움	불필요 (매 프레임 새로 계산)
비트맵 저장	플래시 메모리(PROGMEM) 바이트 배열	레지스터 배열 → 합성 시 조합논리(MUX, 디코더)
물체 수 증가 시	처리 시간 증가	회로 면적 증가 (for = 회로 복제, 외계인 25벌)
갱신 단위	loop() 1회 (명령 수에 따라 가변)	화소 1개/클럭, 1화면 = 8,192클럭 (고정)

2.3 Q.10 RTL에서 확인된 오류

표 3. Q.10 RTL 오류 목록

#	위치	내용	영향
①	225~228줄	대포가 game_tick 조건 없이 매 클럭 이동	키 입력 시 벽까지 이동 (3장)
②	235~237줄	총알이 매 클럭 이동	약 50클럭(1μs) 만에 화면 밖으로 이동, 거의 표시되지 않음
③	225~234줄	이동과 발사가 if ... else if로 연결	이동 중 발사 불가
④	441줄	lcd_data <= (scan_x ^ scan_y); 7비트 결과를 1비트에 대입 (WIDTHTRUNC)	최하위 비트만 남아 시작 화면 패턴이 1화소 격자로 변경 (Q.05: scan_x[2] ^ scan_y[2])
⑤	259~285줄 등	클럭 블록 안 블로킹 대입 shoot_col =, hit_detected = (BLKSEQ 8건)	시뮬레이션과 합성 회로의 동작이 다를 수 있음

⑥	100줄	스프라이트 ROM을 initial 블록으로 초기화	FPGA에서는 동작, 표준 셀 ASIC에서는 초기값 없음 → case문 조합 ROM 필요
---	------	-----------------------------	---

3. Q.11 실습과 좌우 화살표 키 이상 반응

3.1 실습

표 4. Q.11 실행 과정의 문제와 조치

문제	원인	조치
실행 시 libsystemc.so.3.0: cannot open shared object file	Makefile 라이브러리 경로가 lib-linux64, 실제 위치는 /opt/systemc/lib	export LD_LIBRARY_PATH=/opt/systemc/lib:\$LD_LIBRARY_PATH
창이 작아 화면 확인 불가	SDL 창 128×64	SDL_CreateWindow 크기를 512×256으로 변경

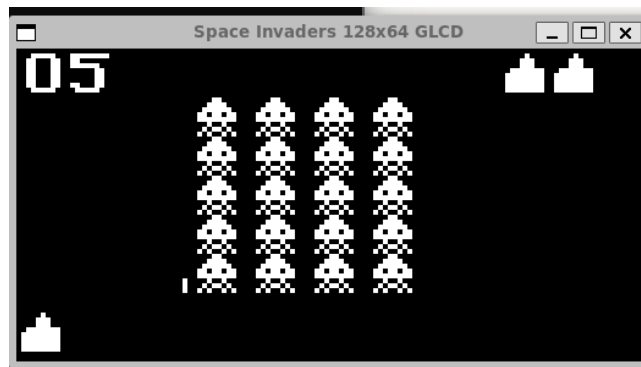


그림 4. Q.11 실행 화면. 대포가 왼쪽 벽에 위치. 실행 중 대포는 항상 양쪽 벽 중 한 곳에 위치함

표 5. Q.11 키 입력 관찰 결과

#	조작	관찰
①	← 또는 → 를 짧게 누름	대부분 반응 없음
②	3회 연속 또는 길게 누름	이동함
③	대포를 중간에 정지	불가. 이동 시 벽까지 이동
④	오른쪽 벽에서 ←, → 동시 입력	이동 없음
④'	왼쪽 벽에서 ←, → 동시 입력	오른쪽 벽까지 이동
⑤	게임 진행 중	정지 후 자동 재개, 재개 시 목숨 3 → 2
⑥	게임 진행 중	외계인 총알과 접촉하지 않은 것으로 보이는 상태에서 대포 소멸, 약 1분 정지 후 재개

3.2 원인 분석

(1) 입력 확인 주기와 이동 주기의 불일치 (①②③)

```
// 테스트벤치 sc_invaders_TB.cpp: 시뮬레이션 시간 16.6ms마다 키보드 상태 확인 (폴링)
wait(16.6, SC_MS);
const UInt8* state = SDL_GetKeyboardState(NULL);
btn_left.write(state[SDL_SCANCODE_LEFT] ? true : false);

// RTL 173~176줄: 버튼이 1인 동안 매 클럭 1칸 이동
if (btn_left && (cannon_x > 7'd1)) cannon_x <= cannon_x - 1'b1;
if (btn_right && (cannon_x < (SCREEN_WIDTH - OBJ_WIDTH - 7'd2))) cannon_x <= cannon_x + 1'b1;
```

표 6. 입력 확인 주기와 이동량

항목	값
테스트벤치 클럭	20ns (50MHz)
키보드 확인 주기	시뮬레이션 시간 16.6ms = 약 830,000클럭
벽에서 벽까지 이동에 필요한 클럭	118클럭 (cannon_x 1 ~ 118)
시뮬레이션 속도 (측정)	약 120만 클럭/초 (SDL 미사용 자동 실행 기준) → 실제 시간보다 수십~수백 배 느림

- ①②: 키보드 확인이 실제 시간으로 수 초 간격이 되어, 그 사이의 짧은 입력은 확인되지 않음
- ③: 입력이 확인되면 버튼이 830,000클럭 동안 1을 유지하므로, 118클럭 만에 벽에 도달

(2) 같은 클럭에 대한 두 논블로킹 대입의 경쟁 (④④')

표 7. 동시 입력 시 cannon_x 변화 (나중 대입 btn_right가 반영됨)

cannon_x	← 조건 (> 1)	→ 조건 (< 118)	다음 값
118 (오른쪽 벽)	참 → 117	거짓	117
117	참 → 116	참 → 118	118 (117 ↔ 118 반복, 정지한 것으로 보임)
1 (왼쪽 벽)	거짓	참 → 2	2
2 ~ 117	참 → -1	참 → +1	+1 (오른쪽 벽까지 이동)

- 같은 클럭에 같은 레지스터로 향하는 두 <= 대입은 모두 대입 전 값(118)으로 계산되며, 소스상 나중 문장의 값이 저장됨

(3) 사망 대기 상태 (⑤⑥)

```
// 피격 시 (RTL 252~258줄)                // 사망 대기 (130~133줄)
inv_bullet_active <= 1'b0;                STATE_DIE: if (die_timer == 6'd60)
player_lives      <= player_lives - 1'b1; next_state = (player_lives == 0) ? STATE_OVER :
STATE_PLAY;
current_state     <= STATE_DIE;            assign draw_cannon = (current_state != STATE_DIE) && ...
```

- ⑤⑥ 모두 대포가 사라지는 사망 대기(STATE_DIE) 상태. 60틱 = 60 × 833,333클럭 = 시뮬레이션 1초 → 실제 약 1분
- 피격 처리에서 current_state를 데이터 처리 블록에서도 구동 → 두 always 블록의 다중 구

동 (합성 불가, Q.13 3번 RTL FPGA_err)

(4) 충돌 판정과 스프라이트의 불일치 (⑥)

키 입력 없이(cannon_x = 59) Q.11 RTL을 Verilator로 자동 실행하여 피격 시점의 외계인 총알 좌표를 기록하였다.

표 8. 피격 판정 시점의 총알 위치와 스프라이트 비트

피격	총알 (x, y)	대포 기준 (열, 행)	스프라이트 행	해당 비트
1	(64, 54)	(5, 0)	00001000	0 (빈칸)
2	(66, 54)	(7, 0)	00001000	0 (빈칸)
3	(65, 56)	(6, 2)	00011100	0 (빈칸)

```
// 충돌 판정: 사각형 범위만 비교 (<= 이므로 가로·세로 9칸)
if ((inv_bullet_x >= cannon_x) && (inv_bullet_x <= cannon_x + OBJ_WIDTH) &&
    (inv_bullet_y >= CANNON_Y) && (inv_bullet_y <= CANNON_Y + OBJ_HEIGHT))
```

- 표시 회로는 스프라이트 비트(sprite_cannon)를 참조하고, 충돌 판정 회로는 9×9 사각형만 참조함. 대포 상단은 대부분 빈칸이므로, 표시상 대포 옆을 지나는 총알도 피격으로 판정됨
- 스프라이트 모양까지 판정하려면 비교 회로가 추가로 필요함 (면적 증가)

4. Q.13 시뮬레이션

4.1 실습

```
$ cd Q.13_SC_Final_PSCE/simulation
$ make # RTL 5종 중 선택 → 심볼릭 링크 생성
$ make lint / build / run / wave / clean
$ VCD_TRACE=NO make build # 파형 기록 끄 (기록 시 20초 실행에 약 150MB)
```

표 9. Q.13 RTL 버전

#	파일	상태
0	Org_wild_cannon	Q.11과 동일 (대포 순간이동)
1	bullet_no_disp	대포 수정, 단발 사격 추가 / 총알 미표시
2	btn_fire	단발 사격 없음 / 발사 버튼 문제
3	FPGA_err	current_state 다중 구동 / FPGA 합성 오류
4	Ok	최종 수정본

- 0번, 4번: make lint, build(약 1분 40초), run 정상 동작. 20초 실행 시 약 110~120프레임 (1프레임 = 8,192클럭 × 100ns = 0.82ms → 실제 시간 대비 약 200배 느림)
- RTL 선택 시 질문 출력 전에 입력하여 선택이 무시됨(Selection out of range) → ln -sf ../space_invaders_engine_glcd/4_...v.0k ../space_invaders_engine_glcd/space_invaders_engine_glcd.v로 직접 연결

4.2 SystemC 테스트벤치 변경점 (Q.11 → Q.13)

표 10. 테스트벤치 변경점

항목	Q.11	Q.13
클럭	스레드에서 직접 생성 (20ns, 50MHz)	sc_clock clk("clk", 100, SC_NS, ...) (100ns, 10MHz)
키보드 입력 위치	테스트벤치 (keyboard_polling_process)	화면 모델 (sc_glcd128x64_TLM::Button_Thread)
키보드 입력 방식	폴링: 16.6ms마다 상태 확인	사건: SDL_KEYDOWN → 1, SDL_KEYUP → 0 (사건 없으면 100ns 대기)
화면 표시	버퍼 저장 후 별도 스레드가 16.6ms마다 표시	매 클럭 화소 1개 표시, (127, 63)에서 갱신 (RTL 프레임에 동기)
테스트벤치 본체	리셋 + 키보드 폴링	리셋 + 프레임 수 출력 (Frame[n])
파형 기록	없음	TB 신호, Verilog 내부 신호 VCD → make wave
DUT 교체	없음	EMULATED_CO_SIM 정의 시 FPGA 에뮬레이션 모델로 교체 (코-에뮬레이션)
빌드	Makefile 1개 (빌드+실행)	lint/build/run/wave/clean 분리, RTL 선택, --trace --timing, clang++

4.3 키보드 입력 보완을 위한 RTL 변경점 (0번 → 4번)

```
// ① 프레임당 1회 버튼 확인: 상태 추가 (2비트 4상태 → 5비트 one-hot 5상태)
localparam STATE_CHK_BTN = 5'b00100;
STATE_PLAY: ... else if ((scan_x == 127) && (scan_y == 63)) next_state = STATE_CHK_BTN;
STATE_CHK_BTN: next_state = STATE_PLAY; // chk_btn <= 1 (1클럭 펄스)
if (chk_btn && btn_left && (cannon_x > 7'd1)) cannon_x <= cannon_x - 1'b1;

// ② 발사 버튼 상승 에지 검출
reg btn_fire_dly; // btn_fire_dly <= btn_fire;
wire btn_fire_pe = btn_fire && !btn_fire_dly;
if (btn_fire_pe && !bullet_active) begin ... end

// ③ 총알 이동을 game_tick에 동기: if (bullet_active && game_tick) ...
// ④ 상태 단일 구동: player_hit <= 1'b1; STATE_PLAY: if (player_hit) next_state = STATE_DIE;
// ⑤ 분주 값: game_tick = (game_tick_counter == 20'd16000); // 기존 833333
```

표 11. RTL 변경점 요약

문제 (0번)	원인	변경 (4번)
대포 순간이동	버튼이 1인 동안 매 클럭 이동	STATE_CHK_BTN: 1프레임(8,192클럭)에 1칸
연사, 시작 화면 건너뛸	버튼 상태(level)에 반응	btn_fire_pe: 상승 에지에만 반응
총알 미표시	매 클럭 이동	game_tick마다 이동
FPGA 합성 오류	current_state 다중 구동	player_hit 플래그로 FSM에 전달
—	클럭 주파수 변경 대응	game_tick 833,333 → 16,000클럭

4.4 파형

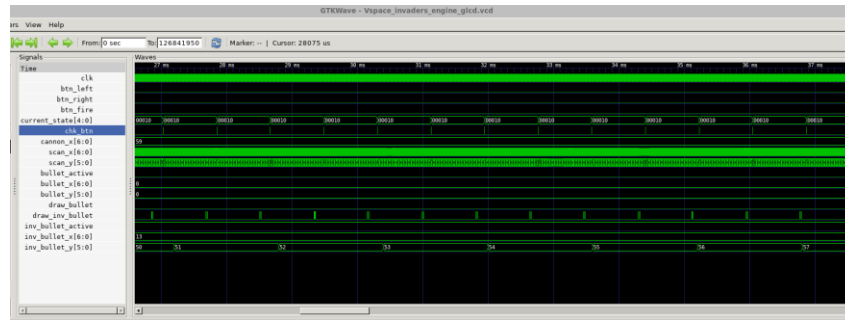


그림 5. Q13 4번 파형 (27~37ms, 키 입력 없음)

표 12. 그림의 신호 간격

신호	간격	계산
chk_btn 펄스	약 0.82ms	8,192클럭 × 100ns (1프레임)
current_state	0.82ms마다 1클럭 동안 00100	화면 끝에서 CHK_BTN 경유
inv_bullet_y +1	약 1.6ms	16,000클럭 × 100ns (game_tick)

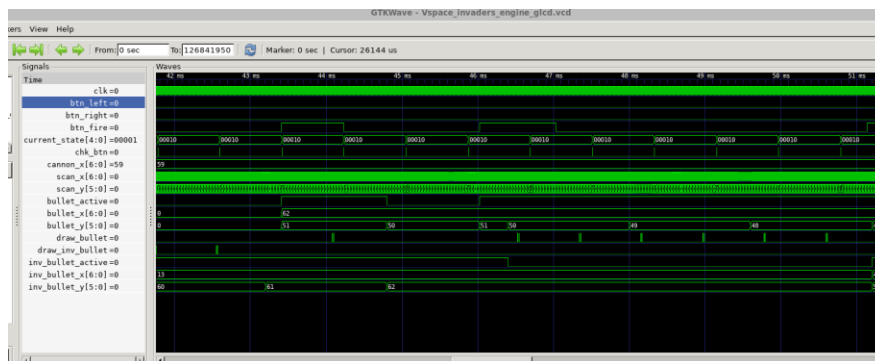


그림 6. Q13 4번 파형 (42~51ms, 발사 버튼 2회 입력)

- btn_fire가 약 0.8ms(수천 클럭) 동안 1을 유지하는 동안 bullet_active는 입력 시작 시점에 1회만 상승 → 상승 에지 검출 동작
- bullet_x = 62 = cannon_x(59) + 3, bullet_y는 51 → 47로 약 1.6ms마다 1 감소
- draw_bullet은 프레임마다 주사 위치가 총알 좌표를 지날 때만 1

4.5 Q.11과 Q.13 최종본 실행 비교

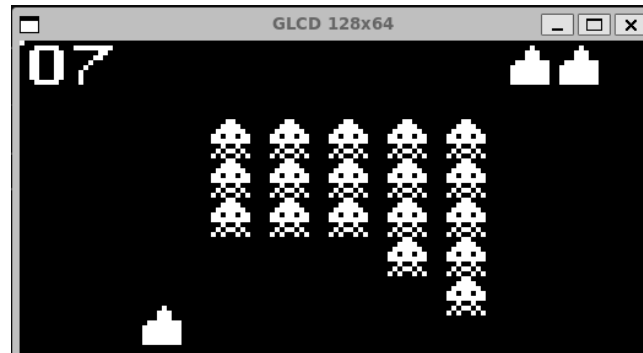


그림 7. Q.13 4번 실행 화면. 대포가 왼쪽 중간에 정지. 점수 07과 제거된 외계인 7마리가 일치

표 13. Q.11과 Q.13 최종본 비교. ④': ← 가 → 보다 먼저 입력되어 ← 만 눌린 화면이 생긴 것으로 추정 (길게 누른 경우는 확인하지 않음)

#	조작	Q.11	Q.13 최종본	관련 변경
①	짧게 누름	대부분 반응 없음	반응함	폴링 → KEYDOWN/KEYUP 사건
②	길게 누름 / 중간	벽까지 이동 / 불가	1칸씩 이동 / 가능	STATE_CHK_BTN
③	정지			
④	오른쪽 벽 동시 입력	정지한 것으로 보임	117 ↔ 118 이동이 화면에 보임	경쟁 구조 유지, 프레임당 1회
④'	왼쪽 벽 동시 입력	오른쪽 벽까지 이동	왼쪽으로 1칸 이동	키입력방식 변경
⑤	발사 1회 / 길게	총알 미표시, 연사	각각 1발, 총알 표시	btn_fire_pe, game_tick
⑦	게임 속도	—	외계인·총알·재시작 모두 빨라짐	game_tick 16,000클럭 (약 52배 빈번)

5. 소프트웨어 언어와 하드웨어 언어

표 14. 총평 관점과 이번 실습의 사례

관점	실습에서 확인한 사례
합성 가능한 RTL	initial ROM (ASIC 합성 불가), current_state 다중 구동 (합성 불가), 클럭 블록 내 블로킹 대입 (BLKSEQ)
a. 비트 단위 객체	[9:0] vga_x → [6:0] lcd_x, 7비트 XOR → 1비트 대입 (WIDTHTRUNC)
b. 다중치 논리 (1, 0, X, Z)	리셋 전 레지스터는 X, 모든 레지스터를 if (!rst_n)에서 초기화, initial 값은 실제 칩에서 보장되지 않음
c. 병렬성과 시간 관리	25개 외계인 판정 회로 동시 동작, for = 회로 복제, <= 동시 갱신과 경쟁(④), 시간 = 클럭 수(game_tick, 0.82ms/1.6ms)
d. 상태와 사건 (edge/event)	posedge clk, btn_fire_pe(상승 에지), STATE_CHK_BTN(프레임 끝), SDL KEYDOWN/KEYUP
SystemC	C++ 클래스 라이브러리 + 병렬 시뮬레이션 엔진: sc_uint<7>, SC_THREAD, posedge_event(), wait(100, SC_NS), sc_clock

버튼을 눌렀을 때 물체가 한 칸 움직이는 기능에 대해 소프트웨어인 아두이노 C++과 하드웨어인 Verilog RTL을 비교해보자. 아두이노에서는 if (!digitalRead(BTN_UP)) 문장이 loop() 한 바퀴에 한 번 실행되고, 1 loop 안에 화면을 그리고 전송하는 시간이 포함되어 있어 탁구 패들이 적당한 속도로 움직였다. 반면, Verilog에서는 같은 역할을 하는 if (btn_left) 문장은 매 클럭 실행되어 Q11(수정 전)에는 방향조작키를 누르면 벽까지 한번에 움직이며 중간에 멈추는 것이 불가했다.

실행 방식도 다르다. 소프트웨어는 문장을 위에서부터 차례로 실행하나, 하드웨어로 구현한 이번 게임에서는 같은 클럭에 동시에 계산되어 두 키를 동시에 누를 때 같은 레지스터를 두고 경쟁하여 왼쪽 벽일 때는 오른쪽 벽까지 한번에 이동하고 오른쪽 벽에서는 움직이지 않았다. 이는 나중에 쓴 오른쪽 이동만 반영되었기 때문으로 확인하였다. 소프트웨어였다면 왼쪽 이동 후 오른쪽 이동이 차례로 실행되어 제자리에 있었을 것이다.

시간을 다루는 방식도 다르다. 소프트웨어는 명령을 실행하는데 걸리는 시간이 게임속도이지만, 하드웨어는 클럭 수를 세서 시간을 만든다. Q13 최종본에서 게임의 속도를 정하는 game_tick의 기준을 833,333클럭 → 16,000클럭으로 변경했을 때 외계인과 총알의 움직임, 사망 후 재시작이 눈에 띄게 빨라졌다.

그림도 소프트웨어의 경우 물체의 위치에 그림을 복사하고, 하드웨어는 화소마다 물체가 있는지 판정한다. 이 때문에 같은 대포라도 그리는 회로는 스프라이트 비트를 화소마다 확인하지만, 충돌을 판정하는 회로는 9×9 사각형만 확인하였다. Q11에서 총알에 맞지 않은 것처럼 보였는데 대포가 사라진 것은, 총알이 스프라이트의 빈칸에 있었지만 사각형 안이었기 때문이다.